GUÍA DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	*Tecnología de Objetos*				
TÍTULO DE LA PRÁCTICA:	*Laboratorio 05*				
NÚMERO DE PRÁCTICA:	5	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*VI*
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**MÍNIMO DE ESTUDIANTES**	----------	**MÁXIMO DE ESTUDIANTES**	*2*
FECHA INICIO:	*14/10/2026*	**FECHA FIN:**	*21/10/2026*	**DURACIÓN:**	*2 horas*
INTEGRANTE (s): *Chipana Jeronimo German Arturo* *Baca Calsin Leonardo Juan José*					
DOCENTE(s): *Edith Giovanna Cano Mamani*					

CONTENIDO DE LA GUÍA

I. SOLUCIÓN

Repositorio:

https://github.com/ChipanaGerman/TecnologiaDeObjetos-Lab-B/tree/main/Lab5

INTRODUCCIÓN

En el desarrollo de software moderno con C++, la expresividad, legibilidad y eficiencia del código son fundamentales, especialmente al diseñar interfaces gráficas de usuario (GUI) interactivas. El estándar C++11 introdujo las funciones anónimas o expresiones lambda, permitiendo encapsular pequeños fragmentos de lógica ejecutable directamente en el lugar donde son invocados, sin sobrecargar la estructura de clases con métodos redundantes.

En paralelo, el framework Qt se posiciona como una de las herramientas multiplataforma más robustas para la construcción de GUIs en C++, basando su arquitectura de comunicación en el paradigma de Señales y Ranuras (Signals & Slots). La integración de funciones lambda con este mecanismo optimiza la manipulación dinámica de componentes visuales (widgets), facilitando la creación de interfaces reactivas, modulares y con una gestión controlada de memoria.

El presente informe documenta el desarrollo práctico del laboratorio, abarcando desde el uso de expresiones lambda puras combinadas con la librería estándar de algoritmos (STL), hasta la construcción modular de una aplicación de gestión de tareas ("Checklist"), demostrando la comunicación desacoplada y la instanciación dinámica de widgets.

OBJETIVOS

Objetivo General

- Analizar, implementar y evaluar el mecanismo de comunicación entre objetos en el SDK de Qt utilizando la arquitectura de Signals & Slots en conjunto con la potencia y flexibilidad de las funciones lambda de C++ moderno en una aplicación GUI basada en widgets.

Objetivos Específicos

- Comprender la sintaxis, bloques de captura ([=], [&], [this]) y ciclo de vida de las expresiones lambda en C++, evaluando su integración con algoritmos de la STL como std::for_each y std::count_if.
- Diseñar e integrar formularios visuales jerárquicos utilizando Qt Designer (MainWindow.ui y componentes desacoplados tipo Task.ui).
- Implementar la instanciación dinámica de widgets personalizados dentro de layouts verticales y gestionar su ciclo de vida y liberación de memoria mediante el método deleteLater().
- Comparar técnica y analíticamente el desempeño, gestión de memoria y evolución de los contenedores secuenciales QVector y QList dentro del ecosistema Qt.

MARCO CONCEPTUAL

- **Funciones Lambda y Bloques de Captura**

Una expresión lambda es una función anónima definida in situ. Su estructura canónica se define como:

$$[\text{capture}](parameters) \rightarrow return_type\ \{body\}$$

- Capture Block (Clausura): Determina cómo el lambda accede a las variables del ámbito circundante:
  - [=]: Captura por valor (copia de solo lectura).
  - [&]: Captura por referencia (permite mutar la variable original externa).
  - [this]: Captura el puntero al objeto de la clase actual, permitiendo invocar sus miembros y métodos directos.

- **Arquitectura Signals & Slots de Qt**

Es un patrón observador de bajo acoplamiento:

- Signal (Señal): Notificación emitida por un objeto QObject cuando ocurre un evento determinado (por ejemplo, el clic de un botón mediante clicked).
- Slot (Ranura): Función miembro o expresión lambda receptora que se ejecuta en respuesta inmediata a la emisión de una señal.
- Macro Q_OBJECT: Macro obligatoria en el encabezado de cualquier clase que utilice el sistema de metaobjetos de Qt para habilitar la introspección, señales y ranuras.

- **Ciclo de Vida y Gestión de Memoria en Qt Widgets**

Qt implementa un modelo jerárquico de relaciones padre-hijo (parent-child). Cuando un QObject es destruido, este se encarga automáticamente de liberar a todos sus hijos en memoria. Para componentes que se desacoplan y eliminan en tiempo de ejecución (como una tarea eliminada de una lista), se emplea deleteLater(), el cual encola la destrucción del objeto en el bucle de eventos (Event Loop) garantizando que no se elimine mientras procesa una señal activa.

Problema Resuelto:

Actividad A: Funciones Lambda

Código

main.cpp* <Select Symbol>

```cpp
#include <QCoreApplication>
#include <QDebug>
#include <vector>
#include <algorithm>

int main(int argc, char *argv[]) {
    QCoreApplication a(argc, argv);

    // 1. Lambda que suma un valor (+3)
    int (*s)(int) = [](int valor) { return valor + 3; };
    int u = 7;
    qDebug() << "Resultado suma lambda:" << s(u); // Imprime 10

    // 2. Lambda para recorrer un vector
    std::vector<int> v = {5, 2, 3, 7, 1, 0, 14, 20};
    qDebug() << "Recorrido del vector:";
    std::for_each(v.begin(), v.end(), [](int valor) {
        qDebug() << valor;
    });

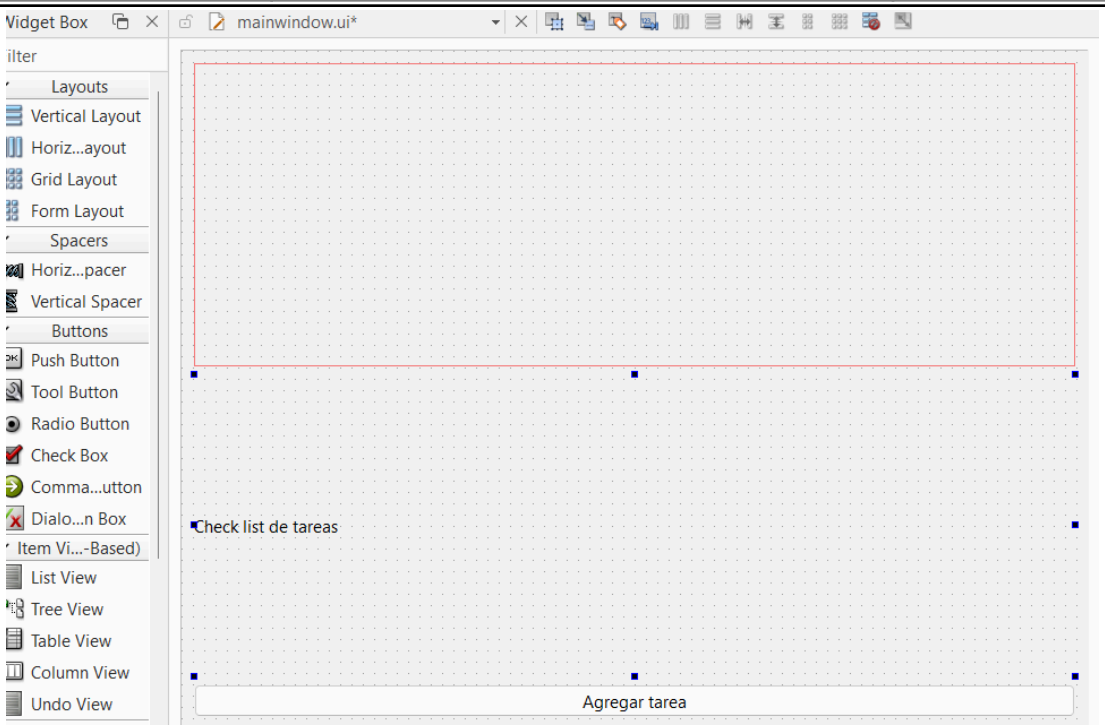
    return 0;
}
```

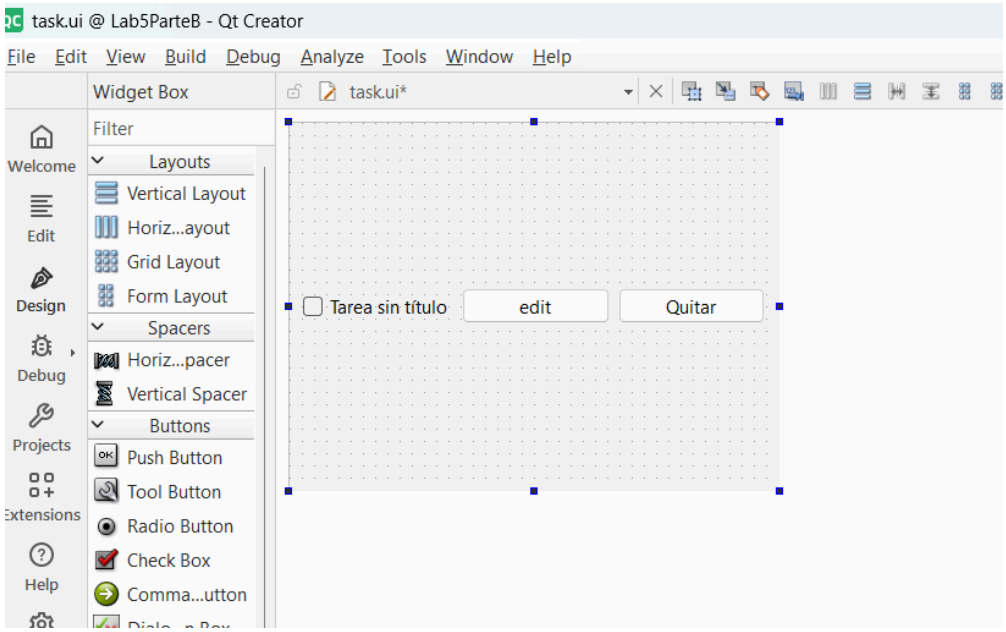
Ejecución

```
11:53:12: Starting C:\Users\User\OneDr
Resultado suma lambda: 10
Recorrido del vector:
5
2
3
7
1
0
14
20
```

Se implementó una expresión lambda anónima mediante un puntero de función para realizar la suma de un valor numérico, demostrando el paso de parámetros y retorno. Asimismo, se utilizó el algoritmo estándar std::for_each combinándolo con una función lambda como predicado de acción, recorriendo de manera eficiente cada elemento del vector sin necesidad de declarar funciones auxiliares externas.

Actividad B: Aplicación Qt Widgets (Checklist de Tareas)


Widget Box
mainwindow.ui*
Layouts
Vertical Layout
Horiz...ayout
Grid Layout
Form Layout
Spacers
Horiz...pacer
Vertical Spacer
Buttons
Push Button
Tool Button
Radio Button
Check Box
Comma...utton
Dialo...n Box
Item Vi...-Based)
List View
Tree View
Table View
Column View
Undo View
Check list de tareas
Agregar tarea


task.ui @ Lab5ParteB - Qt Creator
File Edit View Build Debug Analyze Tools Window Help
Widget Box
task.ui*
Filter
Welcome
Edit
Design
Debug
Projects
Extensions
Help
Layouts
Vertical Layout
Horiz...ayout
Grid Layout
Form Layout
Spacers
Horiz...pacer
Vertical Spacer
Buttons
Push Button
Tool Button
Radio Button
Check Box
Comma...utton
Tarea sin título
edit
Quitar

task.h

```h
#ifndef TASK_H
#define TASK_H

#include <QWidget>

namespace Ui {
class Task;
}

class Task : public QWidget
{
    Q_OBJECT

public:
    explicit Task(QWidget *parent = nullptr);
    ~Task();

signals:
    // Señal para notificar que esta tarea solicita ser eliminada
    void removed(Task* task);

private:
    Ui::Task *ui;
};

#endif // TASK_H
```

task.cpp

```cpp
#include "task.h"
#include "ui_task.h"

Task::Task(QWidget *parent) :
    QWidget(parent),
    ui(new Ui::Task)
{
    ui->setupUi(this);

    // Conexión mediante Lambda: al hacer clic en 'Quitar', emite la señal removed enviando su propio puntero
    connect(ui->removeButton, &QPushButton::clicked, [this]() {
        emit removed(this);
    });
}

Task::~Task()
{
    delete ui;
}
```

La clase Task hereda de QWidget y contiene la macro Q_OBJECT. En su constructor se conecta la señal clicked del botón de eliminación a una función lambda que captura this y emite la señal personalizada removed(Task* task), notificando a la ventana principal qué instancia específica debe destruirse.

mainwindow.h <Select Symbol>

```h
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QVector>
#include "task.h"

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

public slots:
    void addTask();
    void removeTask(Task* task);

private:
    Ui::MainWindow *ui;
    QVector<Task*> mTasks; // Vector para almacenar punteros a las tareas instanciadas
};

#endif // MAINWINDOW_H
```

mainwindow.cpp — MainWindow::removeTask(Task *) -> void

```cpp
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDebug>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
    , mTasks()
{
    ui->setupUi(this);

    // Conexión del botón "Agregar tarea" a su respectivo slot personalizado
    connect(ui->addTaskButton, &QPushButton::clicked, this, &MainWindow::addTask);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::addTask()
{
    qDebug() << "Slot corresponde al clic en el boton ...";

    // 1. Instanciar dinámicamente un nuevo widget Task
    Task* task = new Task(this);

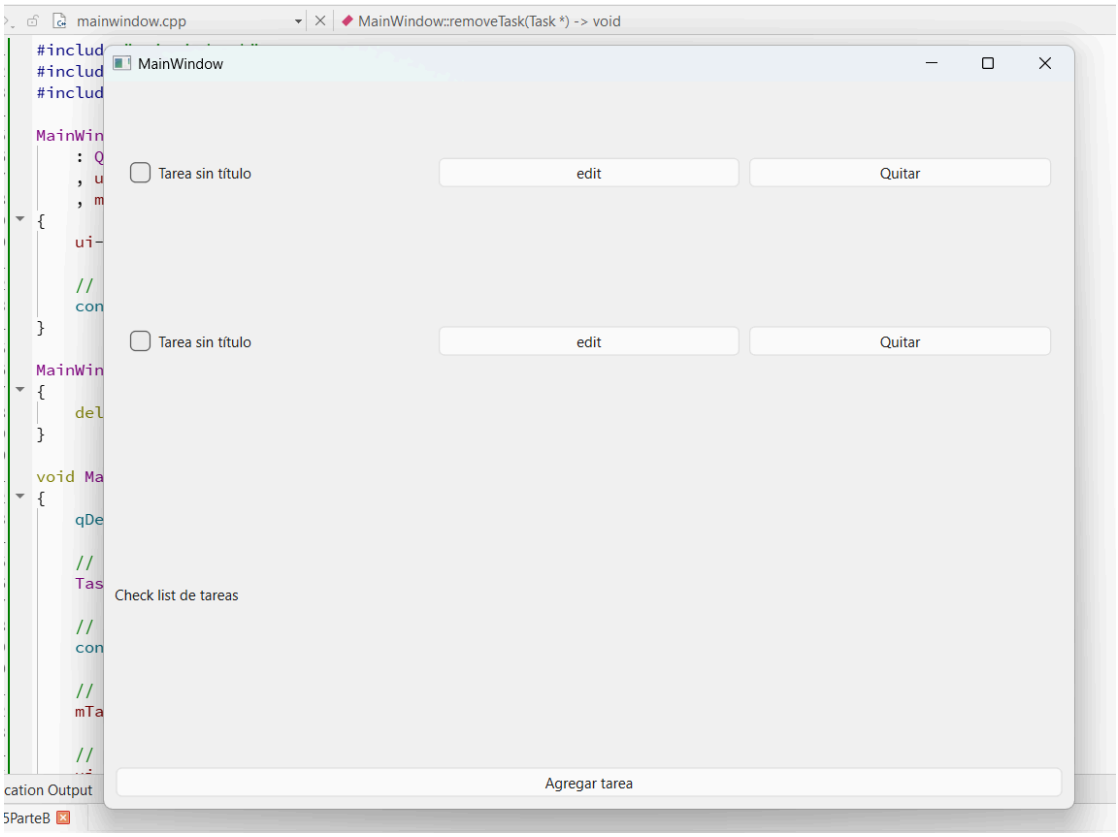
    // 2. Conectar la señal de eliminación del task a la función removeTask del Ma
    connect(task, &Task::removed, this, &MainWindow::removeTask);

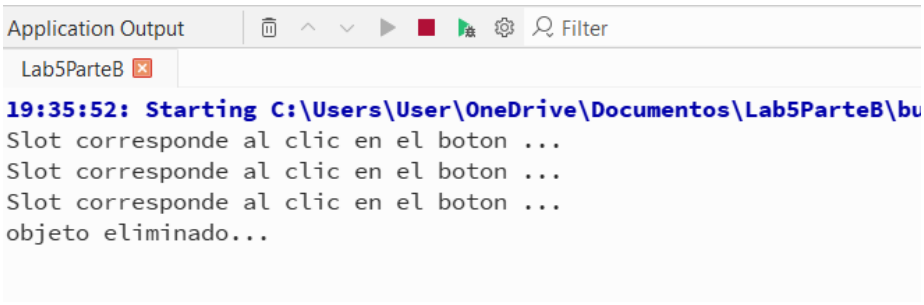
    // 3. Guardar el puntero en el vector de tareas
    mTasks.append(task);

    // 4. Agregar visualmente el widget al vertical layout
    ui->tasksLayout->addWidget(task);
}

void MainWindow::removeTask(Task* task)
{
    qDebug() << "objeto eliminado...";
```

En MainWindow.cpp, el slot addTask() se encarga de crear dinámicamente instancias de Task, añadirlas al contenedor QVector<Task*> y ubicarlas dentro de tasksLayout. A su vez, se vincula la señal removed de la nueva tarea con el slot removeTask(), el cual desacopla el widget de la interfaz visual y libera la memoria mediante deleteLater() evitando fugas de memoria (memory leaks).





Problema Propuesto:

Ejercicio 1: Contar los números impares y menores de 20 de un vector de números enteros generados de forma aleatoria (la solución puede usar el método std :: count_if)

Diseño e Implementación

Se implementó el motor Mersenne Twister std::mt19937 sembrado mediante std::random_device, junto con la distribución uniforme std::uniform_int_distribution<int> configurada en el intervalo [1, 50].

El criterio de filtrado se formuló a través de una expresión lambda que actúa como predicado unario:

```
auto esImparYMenorQue20 = [](int n) {
    return (n < 20) && (n % 2 != 0);
};
```

Dicha lambda es consumida directamente por la función algorítmica std::count_if(numeros.begin(), numeros.end(), esImparYMenorQue20), resolviendo el conteo de forma puramente declarativa sin contadores manuales ni variables de estado mutable.

Código Fuente Completo: Lab5/Ejercicio1/main.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>

using namespace std;

int main() {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<int> dis(1, 50);

    const int TAM = 20;
    vector<int> numeros(TAM);

    for (int i = 0; i < TAM; ++i) {
        numeros[i] = dis(gen);
    }

    cout << "Vector generado:" << endl;
    for_each(numeros.begin(), numeros.end(), [](int n) {
        cout << n << " ";
    });
    cout << "\n\n";

    // Condicion: impar y menor que 20
    auto esImparYMenorQue20 = [](int n) {
        return (n < 20) && (n % 2 != 0);
    };

    int total = count_if(numeros.begin(), numeros.end(), esImparYMenorQue20);

    cout << "Numeros impares y menores a 20:" << endl;
    for_each(numeros.begin(), numeros.end(), [&esImparYMenorQue20](int n) {
        if (esImparYMenorQue20(n)) {
            cout << n << " ";
        }
    });
    cout << "\n\n";

    cout << "Total de elementos que cumplen la condicion: " << total << endl;

    return 0;
}

```

Ejecución

```
Vector generado:
21 46 33 26 23 48 3 32 4 26 9 50 39 37 1 18 40 11 24 28

Numeros impares y menores a 20:
3 9 1 11

Total de elementos que cumplen la condicion: 4
```

Ejercicio 2: Estudio Comparativo entre QVector y QList

Fundamento Teórico y Estructura Interna

Para responder esto con nivel de ingeniería, hay que entender **cómo manejan la memoria**:

A. QVector<T>

- **Cómo funciona por dentro:** Almacena todos los elementos en un **bloque de memoria contiguo** (uno exactamente al lado del otro), idéntico a std::vector de C++.
- **Localidad de caché:** Excelente. El procesador puede cargar múltiples elementos a la memoria caché en una sola operación, haciéndolo extremadamente rápido al recorrerlo.

B. QList<T>

- **En Qt 5 (Histórico):** Funcionaba como un híbrido:
  - Si el tipo de dato era pequeño (igual o menor al tamaño de un puntero, como int o double), los guardaba contiguos.
  - Si el tipo de dato era grande (un objeto o estructura pesada), QList guardaba un arreglo de **punteros** que apuntaban a los objetos dispersos en la memoria Heap. Esto permitía inserciones rápidas al inicio o final (prepend/append), pero destruía la localidad de caché.
- **En Qt 6 (Actualidad):** La arquitectura cambió. Ahora QList se rediseñó para usar memoria contigua y QVector<T> pasó a ser simplemente un alias (typedef) de QList<T>.

Tabla Comparativa: Diferencias, Ventajas y Desventajas

Criterio	QVector<T>	QList<T>

Disposición en Memoria	Siempre contigua en memoria.	En Qt 5 dependía del tamaño de T (punteros u objetos directos). En Qt 6 es contigua.
Acceso a elementos ([] o .at())	Tiempo constante O(1). Máxima velocidad por caché.	Tiempo constante O(1), pero con posible salto de indirección si maneja punteros (Qt 5).
Inserción al final (append / push_back)	Muy rápida (O(1) amortizado).	Muy rápida (O(1) amortizado).
Inserción al inicio (prepend)	Lenta (O(N)), debe desplazar todos los elementos.	Rápida en Qt 5 (reservaba espacio a ambos lados).
Ventajas	1. Consumo de memoria predecible. 2. Compatible con APIs nativas de C/C++ (data()). 3. Óptimo para grandes volúmenes de datos numéricos.	1. Contenedor por defecto y estándar de la API de Qt. 2. Muy versátil para almacenar punteros a Widgets (QList<Task*>).
Desventajas	Costoso para insertar/eliminar al inicio o en el medio.	En Qt 5 causaba fragmentación de memoria con objetos grandes debido a las múltiples asignaciones en Heap.

Código de Demostración Práctica en Qt

Lab5\main.cpp <Select Symbol>

```cpp
#include <QCoreApplication>
#include <QVector>
#include <QList>
#include <QDebug>

struct TareaSimple {
    int id;
    QString nombre;
};

int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);

    qDebug() << "=== DEMOSTRACION: QVector vs QList ===" << Qt::endl;

    // 1. USO DE QVector (Ideal para datos continuos y recorridos rapidos)
    QVector<int> vectorNumeros;
    vectorNumeros << 10 << 20 << 30 << 40 << 50;

    qDebug() << "Elementos en QVector:";
    for (int num : vectorNumeros) {                    c++11 r
        qDebug() << "  Valor:" << num;
    }

    // 2. USO DE QList (El contenedor estandar utilizado por Qt)
    QList<TareaSimple> listaTareas;
    listaTareas.append({1, "Estudiar Qt"});
    listaTareas.append({2, "Completar laboratorio"});
    listaTareas.prepend({0, "Configurar entorno"}); // prepend inserta al inicio

    qDebug() << Qt::endl << "Elementos en QList:";
    for (const TareaSimple &t : listaTareas) {
        qDebug() << "  ID:" << t.id << " - Descripcion:" << t.nombre;
    }

    // 3. Verificacion de compatibilidad con memoria contigua
    qDebug() << Qt::endl << "Tamano de QVector:" << vectorNumeros.size();
    qDebug() << "Tamano de QList:" << listaTareas.size();

    return 0;
}
```

Ejecución

```
12:07:16: Starting C:\Users\User\OneDrive\Documentos\
=== DEMOSTRACION: QVector vs QList ===

Elementos en QVector:
  Valor: 10
  Valor: 20
  Valor: 30
  Valor: 40
  Valor: 50

Elementos en QList:
  ID: 0  - Descripcion: "Configurar entorno"
  ID: 1  - Descripcion: "Estudiar Qt"
  ID: 2  - Descripcion: "Completar laboratorio"

Tamano de QVector: 5
Tamano de QList: 3
```

Ejercicio 3: Crear una interface gráfica (que implemente señales y slots) que muestre una lista de países, al dar clic sobre alguno que se muestre un Label o Text con el idioma y capital correspondiente.

Diseño de la Interfaz Visual (mainwindow.ui)

Se construyó un formulario interactivo derivado de QMainWindow estructurado con un layout vertical (QVBoxLayout) para garantizar el autoajuste dimensional en pantalla:

- **lblTitulo (QLabel):** Instrucción de interfaz orientada al usuario ('Seleccione un país de la lista:').
- **listPaises (QListWidget):** Componente visual scrolleable que almacena los nombres de los países disponibles.
- **frameDetalle (QFrame):** Panel contenedor inferior con bordes redondeados y fondo diferenciado que alberga las etiquetas descriptivas:
  - **lblPais:** Etiqueta en negrita con el nombre del país seleccionado.
  - **lblCapital:** Etiqueta que proyecta la capital correspondiente.
  - **lblIdioma:** Etiqueta que informa el idioma oficial del territorio.

Modelo de Datos y Mecanismo Signals & Slots con Lambdas

Para almacenar la información sin redundancia, se definió la estructura InfoPais conteniendo los campos de capital e idioma, centralizados en la colección indexada QMap<QString, InfoPais> mPaises.

La comunicación reactiva entre la interacción del usuario y la respuesta de la interfaz se consolidó mediante la sintaxis moderna de connect() apoyada en una función lambda:

```
connect(ui->listPaises, &QListWidget::itemClicked, this, [this](QListWidgetItem *item) {
    onPaisSeleccionado(item);
});
```

La expresión lambda captura el puntero [this], invocando al método onPaisSeleccionado() para actualizar el contenido textual de las etiquetas de forma inmediata al detectar el evento de clic.

Código Fuente: Archivo de Cabecera (mainwindow.h)

```h
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QListWidgetItem>
#include <QMap>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

struct InfoPais {
    QString capital;
    QString idioma;
};

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void onPaisSeleccionado(QListWidgetItem *item);

private:
    Ui::MainWindow *ui;
    QMap<QString, InfoPais> mPaises;

    void cargarPaises();
};

#endif // MAINWINDOW_H

```

Código Fuente: Implementación (mainwindow.cpp)

```cpp
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    setWindowTitle("Ejercicio 3 - Informacion de Paises");

    cargarPaises();

    // Conexion con Signal y Slot usando lambda
    connect(ui->listPaises, &QListWidget::itemClicked, this, [this](QListWidgetItem *item) {
        onPaisSeleccionado(item);
    });
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::cargarPaises()
{
    mPaises = {
        {"Peru", {"Lima", "Espanol"}},
        {"Argentina", {"Buenos Aires", "Espanol"}},
        {"Brasil", {"Brasilia", "Portugues"}},
        {"Chile", {"Santiago", "Espanol"}},
        {"Colombia", {"Bogota", "Espanol"}},
        {"Francia", {"Paris", "Frances"}},
        {"Alemania", {"Berlin", "Aleman"}},
        {"Italia", {"Roma", "Italiano"}},
        {"Japon", {"Tokio", "Japones"}},
        {"Canada", {"Ottawa", "Ingles y Frances"}}
    };

    for (auto it = mPaises.begin(); it != mPaises.end(); ++it) {
        ui->listPaises->addItem(it.key());
    }
}

void MainWindow::onPaisSeleccionado(QListWidgetItem *item)
{
    if (!item) return;

    QString nombrePais = item->text();
    if (mPaises.contains(nombrePais)) {
        InfoPais info = mPaises[nombrePais];
        ui->lblPais->setText("Pais: " + nombrePais);
        ui->lblCapital->setText("Capital: " + info.capital);
        ui->lblIdioma->setText("Idioma: " + info.idioma);
    }
}

```

Pruebas de Funcionamiento de la Aplicación

A continuación se muestran las capturas de pantalla de la aplicación ejecutándose en tiempo real, verificando la reactividad ante los clics del usuario:



III. CONCLUSIONES Y RETROALIMENTACIÓN GENERAL

Conclusiones del Laboratorio

- **Eficiencia y legibilidad con Lambdas:** La adopción de funciones lambda permitió conectar eventos visuales (clicked) con acciones específicas en pocas líneas de código, eliminando la necesidad de declarar slots auxiliares triviales en los archivos de cabecera (.h) y mejorando la cohesión del código.
- **Modularidad en la Arquitectura GUI:** Diseñar la clase Task como un widget independiente permitió encapsular sus componentes internos (checkbox y botones), manteniendo a MainWindow como el controlador principal responsable únicamente del layout y de la orquestación del vector de tareas.
- **Robustez en la Comunicación Desacoplada:** El uso de una señal personalizada removed(Task* task) emitida desde el componente hijo hacia la ventana padre demostró ser un patrón de diseño seguro para la destrucción de widgets, evitando dependencias circulares y referencias nulas.
- **Dominio de Contenedores Secuenciales:** El análisis entre QVector y QList evidenció la convergencia que ha tenido el framework hacia el almacenamiento contiguo en memoria, concluyendo que priorizar la localidad de caché es determinante para optimizar la respuesta del hardware y la interfaz.

Retroalimentación General

- **Aspectos Positivos:** La práctica logra vincular de manera orgánica la teoría de C++ moderno (C++11/C++14/C++17) con una aplicación tangible en el desarrollo de software de escritorio mediante Qt. Pasar de ejercicios de consola con algoritmos estándar a interfaces dinámicas consolida significativamente el aprendizaje del paradigma orientado a eventos.

- **Dificultades Superadas:** La sincronización entre la eliminación del elemento en el vector lógico (mTasks.removeOne()), su retiro del layout visual (removeWidget()) y su liberación definitiva de la memoria (deleteLater()) representó el mayor desafío de concurrencia y orden de ejecución, resuelto exitosamente mediante el Event Loop de Qt.
- **Propuestas de Mejora:** Para futuros laboratorios, sería enriquecedor añadir persistencia de datos (por ejemplo, guardar las tareas creadas en un archivo JSON o SQLite) y aplicar animaciones visuales nativas de Qt (QPropertyAnimation) al agregar o quitar elementos del layout.

IV. REFERENCIAS

- https://www.qt.io/
- **QT creator manual**